François Auger, Zhen Luo,
Bruno Feuvrie, and Feng Li

Multiplier-Free Divide, Square Root, and Log Algorithms

"DSP Tips and Tricks" introduces practical design and implementation signal processing algorithms that you may wish to incorporate into your designs. We welcome readers to submit their contributions. Contact Associate Editors Rick Lyons (R.Lyons@ieee.org) or Clay Turner (clay@claysturner.com).

Many signal processing algorithms require the computation of the ratio of two numbers, the square root of a number, or a logarithm. These operations are difficult when using fixed-point hardware that lack dedicated multipliers, such as low-cost microcontrollers, application specific integrated circuits (ASICs), and field programmable gate arrays (FPGAs). This article presents straightforward, multiplier-free algorithms that implement both division and square roots, based on a technique known as dichotomous coordinate descent (DCD) (See [1] as a starting point, where several articles by the same authors are referenced.) We also make available a multiplier-free logarithm algorithm. All these algorithms are based on iterative methods, which compute the successive elements of a sequence of approximate solutions, just like the Gauss-Seidel, Jacobi, and conjugate gradient methods [2]. We begin our discussion with a scheme for computing the ratio of two real numbers.

Digital Object Identifier 10.1109/MSP.2011.941101
Date of publication: 15 June 2011

COMPUTING THE RATIO OF TWO REAL NUMBERS

To compute the ratio of two real numbers, $x = b/a$, the DCD technique performs the minimization of a criterion function $J(x) = ax^2 - 2bx$. As usual, $J(x)$is minimum when its derivative $D(x) = dJ(x)/dx = 2ax - 2b$ equals zero. Since $D(x) = 0$ when $x = b/a$, finding the minimum of $J(x)$is really equivalent to computing b/a. To this aim, the DCD algorithm defines two sequences x_n and $d_n = D(x_n)$, starting with $x_0 = 0$ and $d_0 = -2b$, and a positive real step-size parameter h. At each iteration of the algorithm, x_{n+1} is chosen between $x_n + h$ and $x_n - h$ such that $J(x_{n+1}) < J(x_n)$. This way, $J(x_n)$ will be a decreasing sequence bounded by $-b^2/a$ that will converge, and x_n will go to b/a. Using straightforward computations, one can show that

$$\text{If } d_n < -a\cdot h \text{ then } J(x_n + h) < J(x_n) \quad (1)$$

$$\text{If } d_n > a\cdot h \text{ then } J(x_n - h) < J(x_n). \quad (2)$$

Equations (1) and (2) mean that the correct value of x_{n+1} will be obtained by comparing d_n to $-a\cdot h$ and $a\cdot h$. If $d_n < -a\cdot h$, then $x_{n+1} = x_n + h$ and $d_{n+1} = d_n + 2ah$. If $d_n > a\cdot h$, then $x_{n+1} = x_n - h$ and $d_{n+1} = d_n - 2ah$. If none of these conditions are met, h is divided by two, and the d_n conditions in (1) and (2) are checked again. All these equations require additions and only one multiplication for the computation of $a\cdot h$. But if h is initially chosen as a power of two, this multiplication will be simply implemented using binary right or left shifts. To present it more clearly, Figure 1 shows a computational flow diagram of this real-valued DCD divide algorithm.

The initial value of h is really a key point of this algorithm. In theory, any nonzero value of the initial value of h can be chosen. But for this algorithm to be efficient, a power of two should be preferred. In this case, the product $a\cdot h$ becomes a binary left or right shift, and dividing h by two becomes a right shift. This algorithm then becomes multiplication free and well suited to fixed-point arithmetic on low-cost microcontrollers, FPGAs, or ASICs.

Moreover, for this algorithm to be as efficient as possible, the initial value of h should be chosen as the highest power of two that can be coded with the format of the output variable: suppose for example that a and b are coded using a fixed-point representation of 8 b with 7 b after the decimal point (all numbers are between $-128/128 = -1$ and $+127/128 \approx +1$). If $a = 1/128$, the smallest value of x will be –128 (obtained for $b = -128/128$) and the highest will be +127 (obtained for $b = +127/128$). As a consequence, the result of the division b/a must be coded with at least 8 b before the decimal point, and the number of bits after the decimal point depends on the required accuracy. For example, one can choose to code the output variable of the algorithm, which is an approximation of the ratio b/a, using 9 b before the decimal point and 7 b after the decimal point. The DCD algorithm will then be as efficient as possible if the initial value of h is chosen as 128.

As an example, for $a = 1.357$, $b = 1.853$, an initial value of $h = 16$, and a result accurate to $N_{\text{bits}} = 28$ b, the final result is reached after only 17 iterations. This means that for this example, the DCD real divide algorithm requires only 34 additions to compute

1053-5888/11/$26.00©2011IEEE

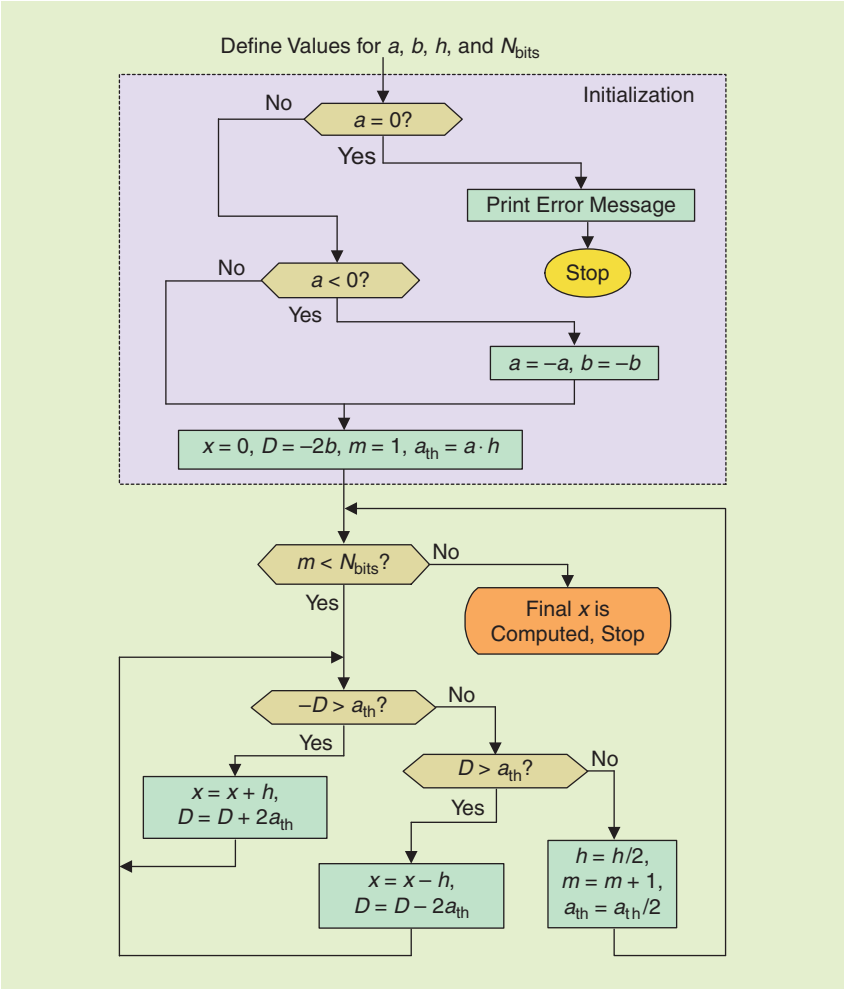
our approximation of $b/a \approx x_{17} = 1.365512$. Figure 2(a) illustrates how fast the points $(x_n, J(x_n))$ reach the minimum of the criterion function, thanks to forward or backward "jumps" of decreasing size. Figure 2(b) shows how the approximation error $e_n = |x_n - b/a|$ decreases with n (the algorithm stops when e_n becomes smaller than $h/2^{N\text{bits}-1} \approx 1.19 \times 10^{-7}$). A comparison of this approximation error e_n with an empirically derived exponential model, $10^{-(n-2)/2}$, shows that the common ratio nearly equals $10^{-1/2}$, which means that only two iterations are necessary to increase the accuracy by one decimal digit.

This divide algorithm clearly belongs to the family of shift and add algorithms [3] and is very close to the linear Coordinate Rotation Digital Computer (CORDIC) algorithm [4]. As the linear CORDIC algorithm, and as another very similar one [5], the DCD algorithm defines two sequences x_n and d_n, such as $2ax_n - d_n$ is kept constant $(2ax_{n+1} - d_{n+1} = 2ax_n - d_n = 2b)$. But the DCD algorithm is deduced from a minimization problem, leading to particular definitions of the sequences x_n and d_n.

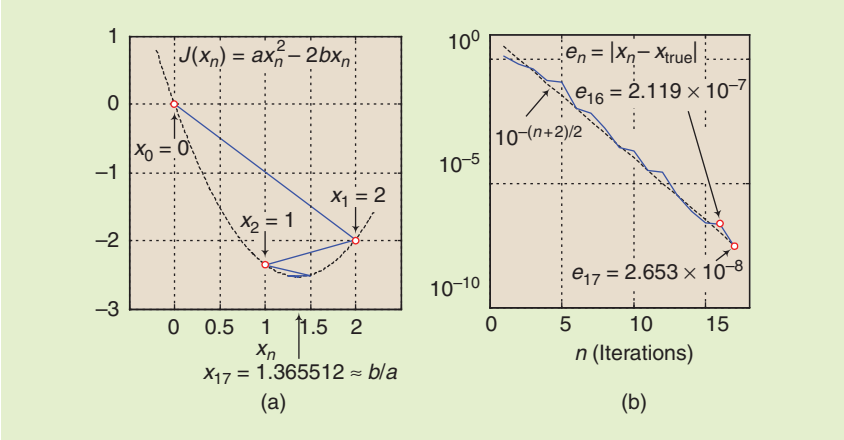
SOLVING A LINEAR SYSTEM $Rx = b$

Moreover, the derivation of the DCD algorithm from a scalar minimization problem also allows the extension of it to a multidimensional one. This will allow finding, by a similar algorithm, the solution of a linear system $Rx = b$, where R is a known symmetric, positive-definite $N * N$ matrix and b is an N-length vector of known real numbers. Many signal processing algorithms, such as parametric spectral analysis tools [6], Kalman state estimators [7], power amplifier linearization [8], and recursive least squares filtering [9] require us to find the solution to such an equation.

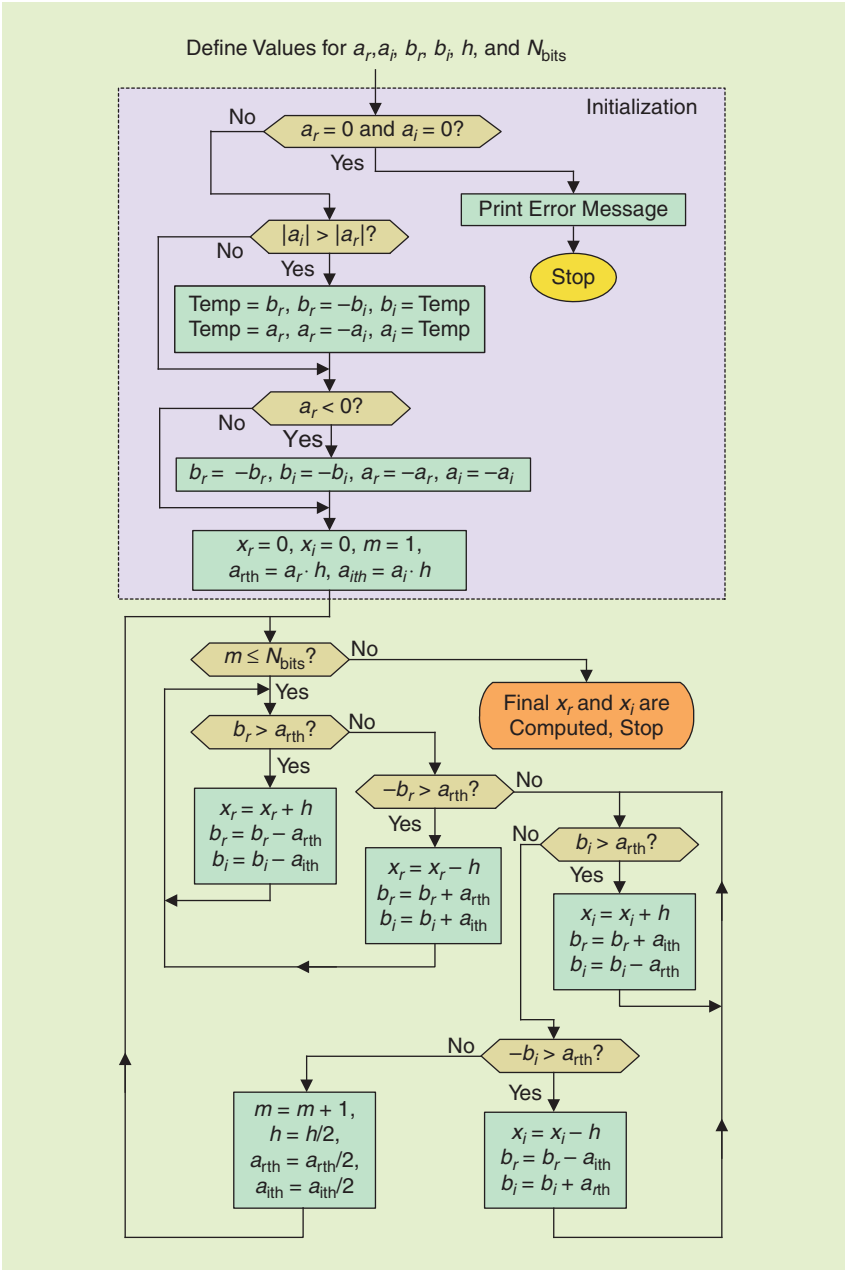
The criterion function to be minimized, in this scenario, will now be defined as $J(x) = x^T Rx - 2x^T b$. This new quadratic criterion is minimum when its gradient $D(x) = dJ(x)/dx = 2Rx - 2b$ equals zero, i.e., when x is the solution of $Rx = b$. To reach this minimum, the DCD algorithm builds a sequence x_k initialized by $x_0 = 0$ and such that $J(x_k)$ decreases and $d_k = D(x_k)$ goes to zero. For this, all the components of the vector x_k are successively considered, checking if either $J(x_k + he_i)$ or $J(x_k - he_i)$ are lower than $J(x_k)$, where h is a step-size parameter and e_i is the ith element of the



[FIG1] **Computational flow diagram of the DCD real divide algorithm.**



[FIG2] **DCD real divide algorithm: (a) path of sequence $J(x_n)$ as a function of x_n and (b) error $e_n = |x_n - b/a|$ versus n (iterations).**



[FIG3] **Computational flow diagram of the DCD complex divide algorithm.**

standard basis (i.e., the ith column of the $N * N$ identity matrix). The resulting final equations are

$$\text{if } d_{i,k} < -hR_{ii}, \text{ then } d_{k+1} = d_k + 2hRe_i \text{ and } x_{k+1} = x_k + he_i \quad (3)$$

$$\text{if } d_{i,k} > hR_{ii}, \text{ then } d_{k+1} = d_k - 2hRe_i \text{ and } x_{k+1} = x_k - he_i, \quad (4)$$

where $d_{i,k} = e_i^T d_k$ is the ith element of the vector d_k and $R_{ii} = e_i^T R e_i$ is the element of R at the ith row and the i th column. If none of these conditions are satisfied, the next component is considered. If none of these conditions are satisfied for the whole components, h is divided by two, and conditions (3) and (4) are checked again for all the successive components. Here again, the sequences x_k and d_k evolve such that $2Rx_k - d_k$ always remains equal to $2b$, with d_k going to zero.

COMPUTING THE RATIO OF TWO COMPLEX NUMBERS

As shown in [10], signal processing algorithms sometimes require to compute the ratio of two complex numbers, $x = b/a$, with $b = b_r + jb_i$ and $a = a_r + ja_i$. For this, the previous algorithm could be used, since a two-dimensional linear system can be deduced from the equation $ax = b$. But this would require multiplications that we try to avoid.

To provide a multiplication-free algorithm, the DCD algorithm proposed in [1] uses another solution that we present here. In our discussion, a_r will be considered as a strictly positive number (a and b can always be simply modified to satisfy this condition without modifying their ratio). The basic principle is to define two complex sequences x_n and $b_n = b_{r,n} + jb_{i,n}$, with $x_0 = 0$ and $b_0 = b$, such that $ax_n + b_n$ is always kept constant

$$ax_{n+1} + b_{n+1} = ax_n + b_n = ax_0 + b_0 = b. \quad (5)$$

If b_n goes to zero, x_n will, by necessity, go to b/a. For example, if b_n can be written as $b_{n+1} + ha$ (h being a step size parameter), such that b_{n+1} is "smaller" than b_n, then

$$\begin{aligned} ax_n + b_n &= ax_{n+1} + b_{n+1} + ha \\ &= a(x_n + h) + b_{n+1} \end{aligned} \quad (6)$$

allowing us to define x_{n+1} as $x_n + h$. If b_{n+1} is not "smaller" than b_n, then three other possibilities can be considered: $b_n = b_{n+1} - h$, $b_n = b_{n+1} + jha$, and $b_n = b_{n+1} - jha$. If any of those possibilities "decreases" b_n, the step size parameter h is divided by two and those four possibilities are checked again. Of course, the words "smaller" and "decreases" are questionable here, because two complex numbers can not be compared. To be more precise, the real and imaginary parts of b_n are alternatively decreased. At even iterations, the real part $b_{r,2k}$ is decreased,

$$\text{if } b_{r,2k-1} > ha_r, \text{ then } b_{2k} = b_{2k-1} - ha \text{ and } x_{2k} = x_{2k-1} + h \quad (7)$$

$$\text{if } b_{r,2k-1} < -ha_r, \text{ then } b_{2k} = b_{2k-1} + ha \text{ and } x_{2k} = x_{2k-1} - h, \quad (8)$$

whereas at odd iterations, the imaginary part $b_{i,2k}$ is decreased

$$\text{if } b_{i,2k} > ha_r, \text{ then } b_{2k+1} = b_{2k} - jha \text{ and } x_{2k+1} = x_{2k} + jh \quad (9)$$

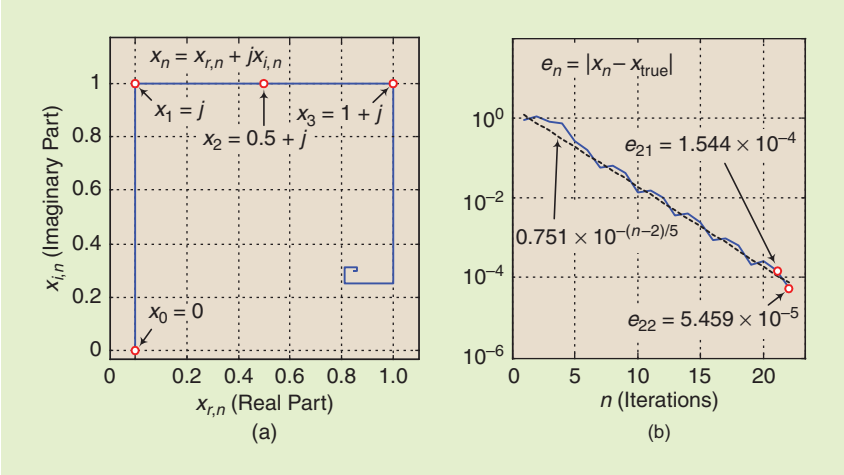
$$\text{if } b_{i,2k} < -ha_r, \text{ then } b_{2k+1} = b_{2k} + jha \text{ and } x_{2k+1} = x_{2k} - jh. \quad (10)$$

As a conclusion, this algorithm adds or subtracts h to the real and imaginary parts of x_n as the real and imaginary parts of b_n decrease. Here also, the key point is that if the initial value of h is chosen as a power of two, this division of two complex numbers will require additions and left or right shifts, but no multiplications. Figure 3 provides a computational flow diagram describing this algorithm.

As an example of this complex divide algorithm, for $a = 3.6 + j3.2$, $b = 2.1 + j3.8$, an initial value of $h = 16$ and a result accurate to $N_{\text{bits}} = 18$ b, the ratio b/a is approximated to $x = 0.8499756 + j\,0.3000488$ with the required accuracy after only 22 iterations (66 additions). Figure 4(a) illustrates how fast the points $(x_{r,n}, x_{i,n})$ spiral to their final values, performing horizontal or vertical "jumps" of decreasing size. A comparison of the approximation error $e_n = |x_n - b/a|$, given in Figure 4(b), with an empirically derived exponential model $0.751 \times 10^{-(n-2)/5}$, indicates that only five iterations are necessary to increase the accuracy of the complex result x by one decimal digit.

COMPUTING SQUARE ROOTS

The DCD principle can have many other applications. For example, it can be applied to design a multiplication-free algorithm solving any quadratic equation written as $x^2 + bx = c$. To show this, we will consider here (without loss of generality) the particular case $b = 0$ and $c > 0$, leading to a square root computation algorithm. Here, again, the "trick" is to define two sequences x_n and c_n, with $x_0 = 0$ and $c_0 = c$, such that $x_n > 0$, $0 \le c_{n+1} < c_n$ and $x_n^2 + c_n$ is kept constant

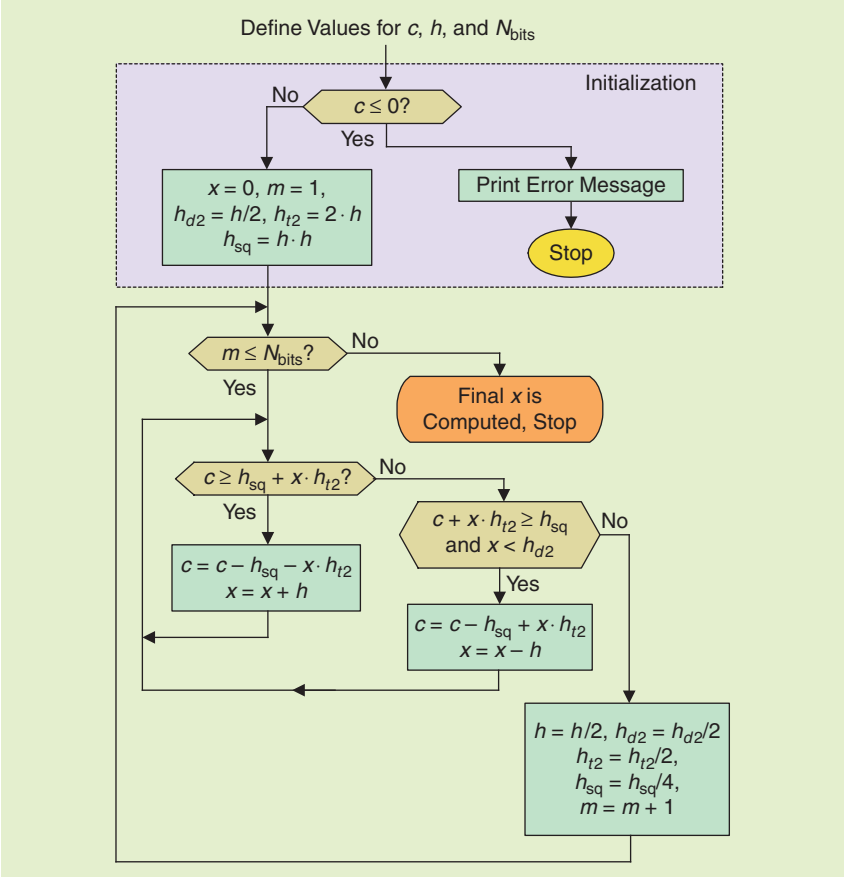


[FIG4] DCD complex divide algorithm: (a) path of sequence x_n, (b) error $e_n = |x_n - b/a|$ versus n (iterations).

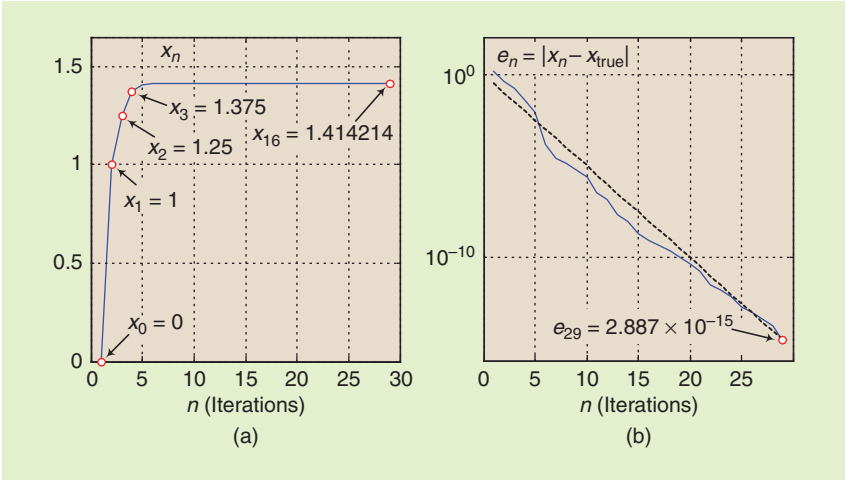
$$x_{n+1}^2 + c_{n+1} = x_n^2 + c_n = x_0^2 + c_0 = c. \quad (11)$$

As c_n goes to zero, x_n will by necessity approach to the nonnegative square root of c. As in all the previous algorithms presented here, we will check if defining x_{n+1} as either $x_n + h$ or $x_n - h$ can lead to $0 \le c_{n+1} < c_n$. The result is

$$\text{if } c_n > h^2 + 2hx_n, \text{ then } c_{n+1} = c_n - 2hx_n - h^2 \text{ and } x_{n+1} = x_n + h \quad (12)$$



[FIG5] Computational flow diagram of the DCD square root algorithm.



[FIG6] DCD square root algorithm: (a) path of sequence x_n and (b) error $e_n = |x_n - c^{1/2}|$ versus n (iterations).

$$\text{if } c_n > h^2 - 2hx_n \text{ and } x_n < h/2,$$
$$\text{then } c_{n+1} = c_n + 2hx_n - h^2$$
$$\text{and } x_{n+1} = x_n - h. \qquad (13)$$

As before, if the initial value of h is chosen as a power of two, this square root computation will be multiplication free. Figure 5 shows a computational flow diagram of this DCD square root algorithm.

As an example, for $c = 2$, an initial value of $h = 2$ and a result accurate to $N_{\text{bits}} = 50$ b, the result is reached after only 29 iterations. Figure 6(a) illustrates how fast the x_n result approaches its final value of 1.414214. Figure 6(b) shows how the approximation error $e_n = |x_n - c^{1/2}|$ decreases with n. The algorithms stops when e_n becomes lower than $h_{\text{init}}/2^{N\text{bits}-1} \approx 3.55 \times 10^{-15}$. A comparison of this error plot with an exponential model $10^{-n/2}$ shows that the common ratio nearly equals $10^{-1/2}$, which means that around two iterations are necessary to increase the x_n result's accuracy by one decimal digit.

COMPUTING LOGARITHMS

Recently, an algorithm to compute the binary logarithm of a positive real number [11] was presented in the "DSP Tips & Tricks" column. This algorithm uses a squaring operation, which requires many elementary arithmetic operations. To this aim, a fast multiplication-free algorithm can also be used. This algorithm is not a DCD algorithm, but it belongs to the more general "shift and add" class. Space restrictions prevent us from including a complete description of this algorithm in this article, but it is available for downloading at the companion Web site. As an example of its performance, the natural logarithm of five is computed using only 20 additions, with an approximation error of $2.3\ 10^{-10}$.

THESE ALGORITHMS ARE PARTICULARLY USEFUL WHEN IMPLEMENTED ON MICROCONTROLLERS, FPGAs, OR ASIC HARDWARE USING FIXED-POINT ARITHMETIC.

CONCLUSION

We have presented computationally simple multiplier-free algorithms for computing 1) the ratio of both real and complex numbers, 2) square roots, and 3) logarithms. These algorithms are particularly useful when implemented on microcontrollers, FPGAs, or ASIC hardware using fixed-point arithmetic.

MATLAB programs that demonstrate the multiplier-free divide and square root algorithms, and a complete description of the logarithm algorithm, are available for download at http://www.signalprocessingsociety.org/publications/periodicals/spm/columns-resources/#tips.

AUTHORS

François Auger (francois.auger@univ-nantes.fr) is an associate professor in the Department of Physical Measurements at the Technological Institute of Saint-Nazaire (University of Nantes, France) and researcher in the IREENA Institute. His current research interests include automatic control of electric machines and power electronics systems, spectral analysis, and time-frequency representation. Dr. Auger is a member of the GdR134 CNRS "Traitement du Signal et des Images."

Zhen Luo (luozhen.lz@126.com) is currently a master student at Guangdong University of Technology, Guangzhou, China. Her research interests include network optimization of WCDMA, and channel allocation in cellular mobile communication systems.

Bruno Feuvrie (Bruno.Feuvrie@univ-nantes.fr) is an assistant professor in the Department of Electrical Engineering at the Technological Institute of Nantes (University of Nantes, France) and researcher in the IREENA Institute. His current research interests include the linearization of high-frequency power amplifiers.

Feng Li (feng.li@etu.univ-nantes.fr) is a Ph.D. student in the IREENA Institute of the University of Nantes, France.

REFERENCES

[1] J. Liu, Y. Zakharov, and B. Weaver, "Architecture and FPGA design of dichotomous coordinate descent algorithms," *IEEE Trans. Circuits Syst. I*, vol. 56, no. 11, pp. 2425–2438, Nov. 2009.

[2] G. Golub and C. Van Loan, *Matrix Computations*, 3rd ed. Baltimore, MD: Johns Hopkins Univ. Press, 1996.

[3] J. Muller, *Elementary Functions, Algorithms and Implementation*, 2nd ed. Cambridge, MA: Birkhäuser, 1997.

[4] R. Andraka, "A survey of CORDIC algorithms for FPGA based computers," in *Proc. 6th Int. Symp. Field Programmable Arrays*, Feb. 1998, pp. 191–200.

[5] A. Guyot, M. Renaudin, B. El Hassan, and V. Levering, "Self timed division and square root extraction," in *Proc. 9th Int. conf. VLSI Design*, 1996, pp. 376–381.

[6] S. Kay and S. Marple, "Spectrum analysis—A modern perspective," *Proc. IEEE*, vol. 69, no. 11, pp. 1380, Nov. 1981.

[7] M. S. Grewal and A. P. Andrews, *Kalman Filtering: Theory and Practice Using MATLAB*, 3rd ed., Wiley-IEEE Press, Sept. 2008.

[8] S. D. Muruganathan and A. B. Sesay, "A QRD-RLS-based predistortion scheme for high-power amplifier linearization," *IEEE Trans. Circuits Syst. II*, vol. 53, no. 10, pp. 1108–1112, Oct. 2006.

[9] S. Haykin, *Adaptive Filter Theory*, 4th ed. Englewood Cliffs, NJ: Prentice-Hall, 2002.

[10] M. Ercegovac and J. Muller, "Design of a complex divider," in *Proc. SPIE Advanced Signal Processing Algorithms, Architectures, and Implementations XIV*, Oct. 2004, pp. 51–59.

[11] C. Turner, "A fast binary logarithm algorithm," *IEEE Signal Processing Mag.*, vol. 27, no. 5, pp. 124–140, Nov. 2010. [SP]